MyCodeSchool

Code review — solar-sync-broker

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 8 — 2026-10-15 — sha `ab9dce1`

Codul revizuit: `solar-sync-broker-api/`, după `d99f677` (EPIC-09 financial), `ce4482a` și `ab630df` (restructurare). Repo-ul are acum numele `solar-sync-broker`.

Tot ce e mai jos se poate rezolva cu ce ai învățat deja: Spring Boot, Spring Data JPA, Spring Testing și Spring Security. Nu ai nevoie de nimic nou.

Verificat prin rulare, pe o copie a codului, pe H2 (codul tău n-a fost atins):

```
mvn -q -DskipTests compile   -> BUILD SUCCESS
mvn test                     -> Tests run: 37, Failures: 0, Errors: 0
mvn test -Dtest='*IT'        -> Tests run: 16, Failures: 0, Errors: 0
CI GitHub (ab9dce1)          -> test, build-push, update-compose, update-platform ✅
```

Aplicația a rulat pe H2 cu trei conturi create prin `/auth/register` și `/auth/login`: un ADMIN, un proprietar (USER) și un USER străin. Casa și bateria (SoC 50%, max 5 kW, randament 100%) le-am introdus direct în bază. Prin API nu se poate, vezi B1 și B8.

Ce e bine ✅

- **Pipeline-ul e verde.** Rulează testele, publică imaginea `sha-ab9dce1` și actualizează ambele compose-uri. E primul run complet pe licență.
- **Restructurarea e corectă.** API-ul e în `solar-sync-broker-api/`, Dockerfile-ul și compose-urile sunt în `-ops/`, iar documentația în `-docs/`.
- **Modulul `financial/` respectă împărțirea Command/Query** din restul proiectului, iar serviciul de citire are `@Transactional(readOnly = true)` la nivel de clasă, exact cum trebuie.
- **`GET /battery` merge pentru proprietar** (`200`). Rezultatul din runda 7 s-a păstrat.

🔴 Critice

B1. Toate endpoint-urile House dau tot 403, iar testul care arăta asta a fost ȘTERS

`HouseController.java:33, :39, :47, :54, :60, :66` — neatins din runda 6.
`HouseControllerIntegrationTests.java` — **șters** în `d99f677` (în runda 7 era comentat).

```
token ADMIN: authorities = [HOUSE_VIEW, HOUSE_MANAGE, BATTERY_VIEW, ...]
GET  /api/v1/houses  -> 403
POST /api/v1/houses  -> 403
```

DE CE: `hasAuthority('house:view')` compară exact șirul de caractere, literă cu literă. Token-ul conține `HOUSE_VIEW`, așa că nimeni nu trece. Testul șters folosea `@WithMockUser(authorities = "house:view")`, adică punea în test exact șirul greșit. De-asta era verde. Când l-ai șters, a dispărut singurul semnal, nu și bug-ul.

💡 De unde vin authority-urile din token? Caută `getAuthorities()` în `User`. 💡 Un test pe un endpoint protejat dovedește ceva doar dacă authority-ul din test vine de unde vine și în realitate.

B2. Oricine își poate crea singur cont de ADMIN

`AuthServiceImpl.java:55, :58` + `AuthReqisterRequest.userType` — deschis din runda 3.

```
POST /api/v1/auth/register {"userType":"ADMIN", ...}  -> 201
login -> token cu HOUSE_MANAGE, BATTERY_CONFIG, BATTERY_COMMAND
```

DE CE: `/auth/**` e `permitAll`, iar rolul vine din corpul cererii. Deci rolul îl alege chiar cel care cere acces. Permisiunile se calculează corect din `userType`, dar valoarea pe care o primesc la intrare e controlată de oricine.

💡 Ce câmpuri din `AuthReqisterRequest` are voie să decidă un vizitator anonim? 💡 Cine ar trebui să poată face un ADMIN? Uită-te la `UserController./add`, l-ai început deja pentru asta (vezi M2).

B3. PATCH pe configurația bateriei răspunde 200, dar nu salvează nimic

`BatteryCommandServiceImpl.java:51` — `@Transactional(readOnly = true)` pe o metodă care SCRIE. Neatins din runda 6.

```
PATCH /houses/1/battery/configuration {"maxChargePowerKw":4.5,"maxDischargePowerKw":4.0}
  -> 200 {"maxChargePowerKw":4.5, "maxDischargePowerKw":4.0}
GET   /houses/1/battery  -> {"maxChargePowerKw":5.0, "maxDischargePowerKw":5.0}
```

DE CE: `readOnly = true` îi cere lui Hibernate să nu trimită modificările în bază (flush mode `MANUAL`). Obiectul din memorie are 4.5, de aceea răspunsul arată bine. Baza n-a primit niciun UPDATE. Iar `BatteryCommandServiceIT.java:102–103` verifică `3.0`, adică exact rezultatul bug-ului (vezi B9).

💡 Pe care metode ale tale ai pus `readOnly`, și ce au ele în comun?

B4. `targetSoc` ajunge în motor ca putere în kW

`BatteryCommandServiceImpl.java:118` — `new BatteryCommand(commandType, targetSoc)`. Neatins din runda 7.

```
POST commands {"commandType":"CHARGE","targetSoc":2}   -> 201 SUCCESS, SoC 50 -> 70
POST commands {"commandType":"CHARGE","targetSoc":90}  -> 201 REJECTED (fără motiv)
```

DE CE: `BatteryCommand` e un record cu parametri poziționali: `(action, powerKw)`. Al doilea argument se asignează după poziție, nu după nume. Așa că „vreau 2%" devine „încarcă cu 2 kW timp de o oră" (+20%), iar „vreau 90%" devine 90 kW, refuzat de validator. Pe deasupra, `catch (Exception)` de la `:124` transformă refuzul într-un `201` fără niciun mesaj.

💡 Ce cere motorul: un procent sau o putere? Decide câmpul pe care îl primește clientul și verifică-l cu un test care se uită la **SoC-ul de după**, nu doar la `201`.

B5. Un străin primește 500 în loc de 403, iar 404 vine cu corpul gol

`BatteryQueryServiceImpl.java:33`, `BatteryCommandServiceImpl.java:111, :148`, `HouseQueryServiceImpl.java:45` aruncă `HouseAccessDeniedHandler`. `GlobalExceptionHandler.java:33` prinde `AccessDeniedExceptions`, o excepție pe care n-o mai aruncă nimeni.

```
GET /houses/1/battery (USER străin)  -> 500
log: HouseAccessDeniedHandler: House access denied
GET /houses/999/battery              -> 404, corp gol
```

DE CE: `@ExceptionHandler` se alege după tipul excepției aruncate, nu după mesajul ei. Tipul aruncat nu are handler, așa că ajunge la Tomcat ca 500. Corpul gol are altă cauză: `:30, :39, :52` trimit `apiErrorResponse.error()`, un câmp pe care nu l-ai setat niciodată, deci `null`.

💡 Ce excepție aruncă serviciile tale și ce excepție prinde handler-ul? 💡 Ce ar trebui să trimiți în corp: un câmp din `ApiErrorResponse` sau tot obiectul?

B6. Proprietarul nu-și poate comanda propria baterie

`AuthServiceImpl.java:99` (`permissionsForType`) + `BatteryController` (`hasAuthority('BATTERY_COMMAND')`). Neatins din runda 7.

```
POST /houses/1/battery/commands (proprietar)  -> 403
GET  /houses/1/battery/commands (proprietar)  -> 403
aceleași cereri, ADMIN                        -> 201 / 200, pe orice casă
```

DE CE: în serviciu ai scris verificarea „ADMIN sau proprietar", dar ea nu apucă să ruleze. `@PreAuthorize` oprește cererea înainte să ajungă în serviciu, iar un USER nu primește niciodată `BATTERY_COMMAND`. Așa că verificarea de proprietar e cod pe care nu-l execută nimeni.

💡 Ce trebuie să poată face un proprietar cu bateria lui? Pune asta pe ramura USER din `permissionsForType`.

B7. Partea financiară dă 403 pentru oricine: aceeași greșeală ca la B1

`FinancialController.java:25, :37` cere `FINANCIAL_VIEW` și `COMMUNITY_VIEW`, care **nu există** în `UserPermissions`. Testele trec doar datorită `@WithMockUser`.

```
GET /houses/1/financial-summary          (ADMIN)  -> 403
GET /houses/community/financial-summary  (ADMIN)  -> 403
FinancialControllerIntegrationTest        -> 3 verzi
```

DE CE: e același mecanism ca la B1. `@WithMockUser(authorities = "FINANCIAL_VIEW")` creează în test un utilizator care nu poate exista în realitate. Testul și controllerul sunt de acord unul cu altul, dar amândoi sunt greșiți față de `/auth/login`. În plus, `recordTransaction` nu e apelat de nicăieri, deci tabela nu se umple decât manual.

💡 Fă testul să pornească de la un login real: `/auth/register` + `/auth/login`, iar token-ul obținut îl pui în header.

B8. Nu există nicio cale de a crea o baterie

Nu există endpoint pentru asta și nici crearea casei nu face o baterie. Neatins din runda 6. Toate endpoint-urile `/battery/**` dau 404 pe orice casă creată prin API.

💡 O casă fără baterie are sens în domeniul tău? Dacă nu, unde ar trebui creată bateria?

B9. Testele care ar putea prinde bug-urile nu rulează în `mvn test`

`pom.xml` nu are `maven-failsafe-plugin`, deci cele 16 teste `*IT` nu rulează în `mvn test` și nici în CI. Neatins din runda 4.

DE CE: Surefire rulează doar clasele care se termină în `Test` / `Tests`. Clasele `*IT` sunt treaba lui Failsafe, la `mvn verify`. Chiar și rulate manual, ele trec: `BatteryCommandServiceIT:102-103` afirmă `3.0`, adică bug-ul B3.

💡 Fie adaugi failsafe, fie le redenumești. Apoi schimbă aserțiunea la valoarea pe care o vrei, nu la cea pe care o primești.

🟡 Importante

- **M1** `FinancialController.java:15` — `/houses` fără `/api/v1`, iar `/{id}/financial-summary` nu verifică proprietarul: oricine are permisiunea vede orice casă.
- **M2** `UserController.java:28` — `hasAuthority('ADMIN')`, dar `ADMIN` e un `UserType`, nu o permisiune, deci răspunsul e mereu 403. În plus, răspunde `201` fără corp.
- **M3** `BatteryCommandRequest.houseId` — e și în body, și în URL. Body `999` + URL `1` → comanda se execută pe casa 1, iar `999` e ignorat.
- **M4** `HouseQueryServiceImpl.java:41` — `IllegalArgumentException` pentru un user care lipsește → 500. În celelalte servicii folosești `UserNotFoundException`.
- **M5** `HouseController.java:41` — `System.out.println(principal)`.

🟢 Cleanups

- **C1** Comentarii de editare rămase în cod: `// Adăugat aici`, `// Corectat la 403`, `// Schimbat din @Component în @Service`.
- **C2** Nume: `AuthReqisterRequest`, `FinancialtransactionResponseToDto`, `HouseAlreadyExistsExcption`, `efficientyPercent`, tabela `bateries`.

- **C3** `HouseQueryServiceImpl` are încă `@Component`, iar câmpurile din `HouseController` / `HouseQueryServiceImpl` nu sunt `private final`.

Ordinea recomandată

1. **B9**: pune testele `*IT` să ruleze. Fără ele nu poți verifica nimic din restul.
2. **B1 + B7**: o singură regulă, un șir de caractere în `@PreAuthorize` trebuie să existe în token. Caută în tot proiectul, nu doar în fișierul din exemplu.
3. **B2, B6**: cine primește ce permisiuni.
4. **B3, B4, B5**: comportamentul bateriei și răspunsurile de eroare.
5. **B8**: crearea bateriei. Abia după asta are rost un feature nou.

Pentru fiecare: **verifică prin rulare, cu token din `/auth/login`**, nu doar cu `mvn test` verde.

Q&A

1. Testul tău financiar trece cu `FINANCIAL_VIEW`. De ce asta nu dovedește că endpoint-ul merge pentru cineva care s-a logat?
2. PATCH-ul răspunde cu valorile noi. De unde vin ele, dacă nu din bază?
3. În `executeCommand` ai o verificare „ADMIN sau proprietar". Pentru ce utilizator ajunge ea să ruleze vreodată?

Runda 7 — 2026-10-06 — sha `edda07a`

- 🔴 B1 — House 403 pentru toți + test comentat. 🔴 **REPETAT**, testul acum șters → runda 8 B1
- 🔴 B2 — `readOnly` pe `updateConfiguration`. 🔴 **NEATINS** → runda 8 B3
- 🔴 B3 — `targetSoc` intră ca `powerKw`. 🔴 **NEATINS** → runda 8 B4
- 🔴 B4 — străin → 500, 404 cu corp gol. 🔴 **NEATINS** → runda 8 B5
- 🔴 B5 — teste `*IT` nerulate. 🔴 **NEATINS** → runda 8 B9
- 🔴 B6 — nicio cale de a crea o baterie. 🔴 **NEATINS** → runda 8 B8
- 🔴 B7 — proprietarul 403 la comenzi. 🔴 **NEATINS** → runda 8 B6
- 🟡 M1 `catch (Exception)` → REJECTED fără motiv 🔴 **NEATINS** (în runda 8 B4) · M5 `getById` nefolosit 🔴 **NEATINS**

Tipar notat atunci: închizi simptomul, nu cauza: testul care arăta bug-ul a fost comentat în loc să fie reparat controllerul.

Runda 6 — 2026-10-01 — sha `5a4da42`

Reacția la runda 5: validări pe DTO, verificare de proprietar pe comenzi, porturi aliniate, cheia JWT scoasă din repo.

- 🔴 B1 — slice-ul House 403 pentru toți (`house:view` vs `HOUSE_VIEW`). 🔴 **NEATINS**, testele comentate → runda 7 B1
- 🔴 B2 — `readOnly = true` pe `updateConfiguration` , PATCH nu salvează. 🔴 **NEATINS** → runda 7 B2
- 🔴 B3 — `GET /battery` 500 LazyInit pentru proprietar. ✅ **ÎNCHIS** (`edda07a`)
- 🔴 B4 — `GET /battery/commands` 500 (LazyInit + repository `null`). ✅ **ÎNCHIS**
- 🔴 B5 — 16 teste `*IT` nerulate. 🔴 **NEATINS** → runda 7 B5
- 🔴 B6 — nicio cale de a crea o baterie. 🔴 **NEATINS** → runda 7 B6
- 🔴 B7 — SoC teleportat, motorul neapelat. 🟡 **PARȚIAL** — motorul e apelat, dar cu procentul pe post de kW → runda 7 B3
- 🔴 B8 — `AccessDeniedExceptions` -> 404 cu corp gol. 🟡 **MUTAT** — handler pe 403 pentru o excepție pe care nu o mai aruncă nimeni → runda 7 B4
- 🔴 B9 — nimeni nu poate comanda bateria. 🟡 **PARȚIAL** — ADMIN poate, proprietarul nu → runda 7 B7
- 🔴 B10 — `Telemetry` fără `@Entity` . ✅ **ÎNCHIS**
- 🟡 M6 — `@InjectMocks` + `new` manual. ✅ **ÎNCHIS** · M7 `@Service` ✅ · M1–M5 🔴 **NEATINSE**
- 🟢 C3 importuri moarte ✅ (în fișierele atinse) · C5 comentarii de editare 🔴 **REPETAT** · C1, C2, C6–C9 **NEATINSE**

Runda 5 — 2026-09-09 — sha `cc74f0c`

- 🔴 B1 — `BatteryCommandServiceTests:35` nu compila. ✅ **ÎNCHIS** (`236471d`)
- 🔴 B2 — `GET /battery` 500 LazyInit. → runda 6 B3, ✅ închis în runda 7
- 🔴 B3 — `BATTERY_COMMAND` / `BATTERY_CONFIG` neacordate. 🟡 **PARȚIAL** — doar ADMIN
- 🔴 B4 — `targetSoc` scris direct în SoC, motorul mort. 🟡 → runda 7 B3

- 🔴 B5 — zero ownership pe comenzi. 🟡 **PARȚIAL**
- 🔴 B6 — `BatteryCommandRequest` fără validări. ✅ **ÎNCHIS** (`0165892`)
- 🔴 B7 — `Telemetry` fără `@Entity` . ✅ închis în runda 7

Runda 4 — 2026-08-19 — sha `e21d7c9`

- 🔴 B1 — `maven-failsafe-plugin` lipsă. 🔴 **NEATINS** → runda 8 B9
- 🔴 B2 — logica din epic-5 fără apelanți. 🟡 **MUTAT**
- 🔴 B3 — clamp SoC inaccesibil (validator vs serviciu). 🔴 **NEATINS**
- 🔴 B4 — `@NotNull` pe `double` primitiv. 🔴 **NEATINS**

Runda 3 — fundația (auth / house / battery)

- 🔴 B1 — oricine se poate înregistra ca ADMIN. 🔴 **NEATINS** → runda 8 B2
- 🔴 B2 — `battery:view` vs `BATTERY_VIEW` . ✅ **parțial închis**
- 🔴 B3 — `GlobalExceptionHandler` cu body `null` . 🔴 **NEATINS** → runda 8 B5
- 🔴 B4 — `UserController./add` mort. 🔴 **NEATINS** → runda 8 M2
- 🔴 B5 — `ddl-auto` / Flyway. ⏸ amânat, migrările nu sunt încă predate